Shifted Binary Search

AlgoExpert · Hard · Searching

The problem

- Find target's index in a sorted array of distinct ints rotated by some amount; -1 if absent.

Intuition

- In a rotated sorted array, one half is always sorted; check if the target lies in it.

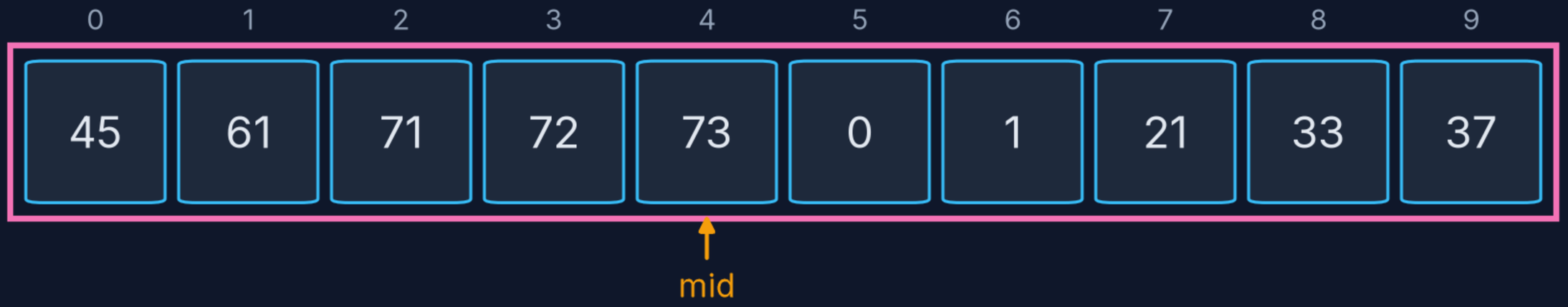
Brute force vs optimal

- Linear scan → O(n)
- Find the pivot, then binary search → O(log n)
- One-pass: identify the sorted half → O(log n)

Approach

- 1. Find the middle
- 2. Decide which half is sorted
- 3. Check whether the target is in that half
- 4. Recurse into the right half

Worked example




```
left half sorted; 33 not in it → go right
```

mid 73: go right

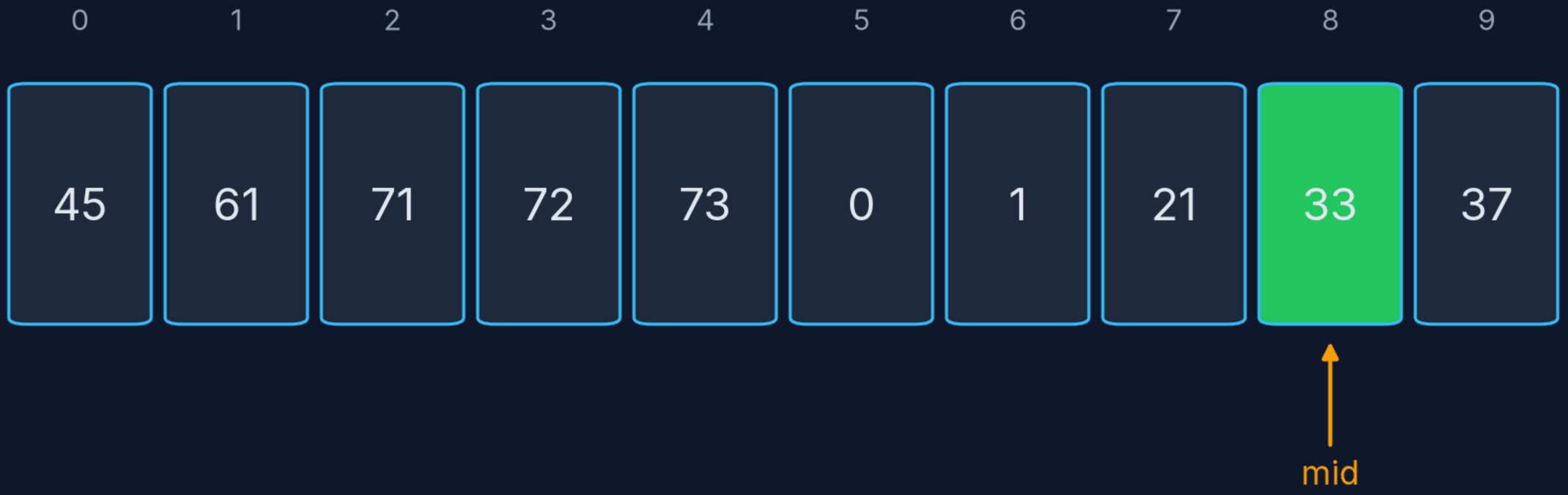
Worked example




```
left half sorted; 33 not in it → go right
```

mid 21: go right

Worked example



Found at 8

Java solution

```java
public class ShiftedBinarySearch {
    public static int shiftedBinarySearch(int[] a, int t) {
        int lo = 0, hi = a.length - 1;
        while (lo <= hi) {
            int mid = (lo + hi) >>> 1;
            if (a[mid] == t) return mid;
            if (a[lo] <= a[mid]) { if (t >= a[lo] && t < a[mid]) hi = mid - 1; else lo = mid + 1; }
            else { if (t > a[mid] && t <= a[hi]) lo = mid + 1; else hi = mid - 1; }
        }
        return -1;
    }
}
```

Complexity

- Time: O(log n)
- Space: O(1)

Edge cases & pitfalls

- Use ≤ when comparing a[lo] and a[mid]
- Duplicates break the guarantee
- Not found → −1

Interview tips & follow-ups

- Follow-up: with duplicates

Recap

- In a rotated sorted array, one half is always sorted; check if the target lies in it.
- Complexity: O(log n)